

作業: GIT(III): git 結構 _ 合併與修改

1. 練習:

MD5

```
shiro@LAPTOP-RPUK0HCC:/tmp$ date > date.txt
shiro@LAPTOP-RPUK0HCC:/tmp$ md5sum date.txt
229a8c2fcfb6fbb179eecd20304d4e10  date.txt
shiro@LAPTOP-RPUK0HCC:/tmp$ date > date.txt
shiro@LAPTOP-RPUK0HCC:/tmp$ md5sum date.txt
e2df34b41bab287da401a34761e2fe2a  date.txt
```

產生不同的 MD5 校驗碼

```
shiro@LAPTOP-RPUK0HCC:/tmp$ md5sum date1.txt date2.txt date3.txt
421ac030553057c64149974192657c5f  date1.txt
83ca65bbbd8870b231a8455bc75ec8cc  date2.txt
77f912ffc25113d5337b4104c824a9c8  date3.txt
```

一次對多個檔案產生 MD5 校驗碼

```
shiro@LAPTOP-RPUK0HCC:/tmp$ md5sum date.txt date_copy.txt
e2df34b41bab287da401a34761e2fe2a  date.txt
e2df34b41bab287da401a34761e2fe2a  date_copy.txt
```

兩個檔案如果名稱不同，但內容相同，還是會產生相同的 MD5 校驗碼

```
shiro@LAPTOP-RPUK0HCC:/tmp$ md5sum date1.txt date2.txt date3.txt > date.md5sum
shiro@LAPTOP-RPUK0HCC:/tmp$ md5sum -c date.md5sum
date1.txt: OK
date2.txt: OK
date3.txt: OK
```

產生每個檔案的 MD5 校驗碼並且儲存起來

```
shiro@LAPTOP-RPUK0HCC:/tmp$ date | md5sum -
dec0913f225ed1c00f8bc69bd2437fe3  -
```

可以從標準輸入讀取資料來計算 MD5 校驗碼

SHA1

```
shiro@LAPTOP-RPUK0HCC:/tmp$ sha1sum date.txt
a47c35d25a70fdbcf6c9e9861f1a8952be1e45c  date.txt
```

產生 SHA1 校驗碼

```
shiro@LAPTOP-RPUK0HCC:/tmp$ sha1sum date1.txt date2.txt date3.txt
6ae64cb3c206bb0d83764b6773cf1048db7f9b4f  date1.txt
a83f4bba64da1e9fda7ea0fee41dfff4957bda994  date2.txt
b0a51e45368d57cb14f559d41b2de11b5b9a5f72  date3.txt
```

一次產生多個檔案的 SHA1 校驗碼

```
shiro@LAPTOP-RPUK0HCC:/tmp$ sha1sum date.txt date_copy.txt
a47c35d25a70fdb8f6c9e9861f1a8952be1e45c  date.txt
a47c35d25a70fdb8f6c9e9861f1a8952be1e45c  date_copy.txt
```

SHA1 校驗碼同樣只依據檔案內容來計算，與檔名無關

```
shiro@LAPTOP-RPUK0HCC:/tmp$ sha1sum date1.txt date2.txt date3.txt > date.sha1sum
shiro@LAPTOP-RPUK0HCC:/tmp$ sha1sum -c date.sha1sum
date1.txt: OK
date2.txt: OK
date3.txt: OK
```

驗證檔案的 SHA1 校驗碼時，也可以使用 `-c` 參數

2.

(a) 何謂 SHA-1 編碼

SHA-1 (Secure Hash Algorithm 1) 是一種雜湊 (hash) 演算法，會把任意長的輸入資料映射成固定長度的位元摘要 (digest)。

SHA-1 輸出的長度是 160 位元 (20 bytes)，用十六進位表示時長度為 40 個字元 (例如 `da39a3ee5e6b4b0d3255bfef95601890afd80709` 這類格式)。

性質：

1. 輸入→輸出單向性 (難以從 digest 還原原始資料)。
2. 固定長度 (不論原始資料多大，輸出長度一樣)。
3. 碰撞 (collision) 風險存在：不同輸入理論上可能得到相同 hash，實務上 SHA-1 被發現有可行碰撞攻擊，已被視為弱化，許多系統逐漸轉向 SHA-256 等更安全的演算法。

(b) SHA-1 在 Git 中的用途

唯一識別 (content-addressable ID)：Git 以物件的 SHA-1 當作該物件的 ID (也就是我們俗稱的 commit hash / blob hash / tree hash / tag hash)。這使得相同內容必然對應相同的 ID，便於去重 (dedup) 與快取。

完整性驗證：因為物件的 SHA-1 是由物件內容計算出來的，Git 在讀寫物件時可檢查 SHA-1 是否匹配，保證內容未被破壞或篡改。

內容導向儲存 (content-addressable storage)：Git 的資料庫 (`.git/objects`) 以 hash 分散存放，取用時直接用 SHA-1 找到對應物件。

版本歷史參照：commit hash (SHA-1) 就是版本的指紋，用來引用、比較與合併不同版本。

實作細節：在計算物件的 SHA-1 時，Git 並不是直接對檔案內容做 hash，而是對加了 header 的內容做 hash，例如：`"<type> <size>\0<content>"`

例如對一個 blob (檔案內容) 會先組成 `"blob 14\0<14 bytes of content>"` 然後再做 SHA-1，這能保證不同類型或長度的物件不會產生相同 hash。

3. 說明 git 結構的四大天王：

1. commit 物件：

- Commit 是 Git 的歷史節點，主要欄位典型內容：
 - tree <tree-sha>：該 commit 對應的樹（整個專案的快照）
 - parent <parent-sha>（可有多個，merge 會有多個 parent）
 - author <name> <email> <timestamp>
 - committer <name> <email> <timestamp>
 - 空行後的 commit 訊息（message）
- Commit 的 SHA-1 是對這整個 commit 物件（header + 以上內容）做 hash，commit id 就是版本號。
- Commit 與 parent 形成有向無環圖(DAG)，代表歷史鏈（可回溯、比較、merge）。

2. Tree 物件：

- 表示一個目錄（folder）的 **列表**，每個 entry 包含：
 - 檔案模式（mode，例如普通檔案 100644、可執行 100755、子目錄 040000）
 - 檔名（filename）
 - 指向的物件 SHA（blob 或另一個 tree）
- Tree 描述某一個時刻某個目錄的結構；整個專案的根目錄對應一個 **root tree**。
- Tree 本身也是一個物件（"tree <size>\0<entries>" 被 SHA-1），因此若目錄內容不變，tree 的 SHA-1 也不變。

3. Blob 物件：

- 內容：原始檔案 bytes（例如某個 .c、.txt 的內容）。
- 無檔名與時間等 metadata（這些由 tree 或 refs 管理）。
- Git 存放方式：blob 物件的內容被 prefix "blob <size>\0" 後做 SHA-1，該 SHA-1 就是這個檔案內容的 ID。
- 功能：同內容的檔案在 repo 中只有一份（去重）。

4. Tag 物件：

- 兩種常見形式：
 - 1.輕量(lightweight)tag：只是建立一個 ref(指向 commit 的 pointer)，不是物件。
 - 2.Annotated tag（標註標籤）：是一個真正的 Git 物件（tag），包含：

- object <sha> (被標記的物件)
- type <type> (object 的類型，通常是 commit)
- tag <tagname>
- tagger <name> <email> <timestamp>
- 標籤訊息 (可包含簽章)
- Annotated tag 可以被 GPG 簽名，是發佈版本 (release) 常用方式

4. 練習 git-learn-git.PDF "合併分支" (Page 145-149)

為了展示結果，在適當步驟，使用 `git log --graph` 指令來呈現 git 樹狀結構

```
git add hello.html
git commit -m "Add hello.html"
[master (root-commit) 0ad61f9] Add hello.html
 1 file changed, 1 insertion(+)
 create mode 100644 hello.html
shiro@LAPTOP-RPUKOHCC:/tmp/git1018$ git log --graph
* commit 0ad61f9c899f32bb9196cabd545444b38289d9b3 (HEAD -> master)
   Author: Shiro <cyhuang1006@gmail.com>
   Date:   Sun Nov 2 22:25:06 2025 +0800

    Add hello.html
```

建立第一個檔案並提交到 master 分支

```
shiro@LAPTOP-RPUKOHCC:/tmp/git1018$ git branch cat
git checkout cat
Switched to branch 'cat'
```

建立新分支 cat 並切換

```

shiro@LAPTOP-RPUK0HCC:/tmp/git1018$ vi cat1.html
git add cat1.html
git commit -m "Add cat1.html"
[cat 3198d74] Add cat1.html
1 file changed, 1 insertion(+)
create mode 100644 cat1.html
shiro@LAPTOP-RPUK0HCC:/tmp/git1018$ vi cat2.html
git add cat2.html
git commit -m "Add cat2.html"
[cat ba06dda] Add cat2.html
1 file changed, 1 insertion(+)
create mode 100644 cat2.html
shiro@LAPTOP-RPUK0HCC:/tmp/git1018$ git log --graph
* commit ba06dda10bcaa501843d0b99723355e58dbde565 (HEAD -> cat)
| Author: Shiro <cyhuang1006@gmail.com>
| Date: Sun Nov 2 22:27:37 2025 +0800
|
| Add cat2.html
|
* commit 3198d74d682f47c04cdfeeb289e6271afc9950c3
| Author: Shiro <cyhuang1006@gmail.com>
| Date: Sun Nov 2 22:26:41 2025 +0800
|
| Add cat1.html
|
* commit 0ad61f9c899f32bb9196cabd545444b38289d9b3 (master)
| Author: Shiro <cyhuang1006@gmail.com>
| Date: Sun Nov 2 22:25:06 2025 +0800
|
| Add hello.html

```

在 cat 分支中新增並提交兩次

```

shiro@LAPTOP-RPUK0HCC:/tmp/git1018$ git checkout master
git merge cat
git log --graph
Switched to branch 'master'
Updating 0ad61f9..ba06dda
Fast-forward
 cat1.html | 1 +
 cat2.html | 1 +
 2 files changed, 2 insertions(+)
 create mode 100644 cat1.html
 create mode 100644 cat2.html
* commit ba06dda10bcaa501843d0b99723355e58dbde565 (HEAD -> master, cat)
| Author: Shiro <cyhuang1006@gmail.com>
| Date: Sun Nov 2 22:27:37 2025 +0800
|
| Add cat2.html
|
* commit 3198d74d682f47c04cdfeeb289e6271afc9950c3
| Author: Shiro <cyhuang1006@gmail.com>
| Date: Sun Nov 2 22:26:41 2025 +0800
|
| Add cat1.html
|
* commit 0ad61f9c899f32bb9196cabd545444b38289d9b3
| Author: Shiro <cyhuang1006@gmail.com>
| Date: Sun Nov 2 22:25:06 2025 +0800
|
| Add hello.html

```

切回 master 並合併 cat

5. 建立另一個新 git 專案目錄，

練習 git-learn-git.PDF "另一種合併方式（使用 rebase）" (Page 168-)

為了展示結果，在適當步驟，使用 `git log --graph` 指令來呈現 git 樹狀結構

(a) 建立 master, cat, dog 分支，產生 page 168 圖的分支結構

(b) 在 cat 中新增 cat1.html, cat2.html

```
shiro@LAPTOP-RPUKOHCC:/tmp/git_rebase$ vi cat1.html
# 內容 : cat1 學號 Hello
git add cat1.html
git commit -m "Add cat1.html"

vi cat2.html
# 內容 : cat2 學號 Hello
git add cat2.html
git commit -m "Add cat2.html"
[cat 6eadbac] Add cat1.html
1 file changed, 1 insertion(+)
create mode 100644 cat1.html
[cat 38b3ca5] Add cat2.html
1 file changed, 1 insertion(+)
create mode 100644 cat2.html
```

(c) 切到 cat，執行 git rebase dog

```
shiro@LAPTOP-RPUKOHCC:/tmp/git_rebase$ git checkout cat
git rebase dog
Already on 'cat'
Successfully rebased and updated refs/heads/cat.
```

(d) rebase 前的 git log --graph

```
shiro@LAPTOP-RPUKOHCC:/tmp/git_rebase$ git log --graph --oneline --all
* 38b3ca5 (HEAD -> cat) Add cat2.html
* 6eadbac Add cat1.html
| * 4eb28a8 (dog) Add dog2.html
| * e9aac3a Add dog1.html
|/
* 19331f3 (master) Add hello.html
```

rebase 後的 git log --graph

```
shiro@LAPTOP-RPUKOHCC:/tmp/git_rebase$ git log --graph
* 1795f91 (HEAD -> cat) Add cat2.html
* f67a9ea Add cat1.html
* 4eb28a8 (dog) Add dog2.html
* e9aac3a Add dog1.html
* 19331f3 (master) Add hello.html
```